Configuring and Installing PETSc for Underworld

Romain Beucher[1]

[1] Australian National University

DOI https://doi.org/10.6084/m9.figshare.33193509

Keywords petsc, installation, Underworld Code

Beucher (2022)

The following is compatible with Ubuntu 20.04 under ***Windows*** 10/11 WSL 2 (***Windows*** Subsystem for Linux).

PETSc, the Portable, Extensible Toolkit for Scientific Computation is the main dependency required for building Underworld. In the following, I will describe my current workflow for configuring and installing PETSc on Linux. The instructions are also valid for the Windows Subsystem for Linux (WSL2) using the Ubuntu 20.04 app.

1. Build dependencies

The minimum required to build PETSc is a C compiler (typically GCC), and a couple of utilities (Make, Python3). We will also need a C++ and a Fortran compiler to build the External packages that are part of the toolkit used by Underworld.

- wget (needed to download the source code, could also use git)
- gcc (C Compiler, required by PETSc)
- g++ (C++ Compiler, required to compile External Packages)
- gfortran (Fortran Compiler, required to compile External Packages)
- make (needed to build the source)
- python3 (needed to configure the build)

Ubuntu Linux (also under WSL2):

...and other Debian based distribution)

```
sudo apt-get update
sudo apt-get install \
    make \
    gcc \
    python3 \
    gfortran \
```

PUBLISHED Jan 20, 2022 ARTICLE UWTN 2022-001 VERSION 1.0.0 LICENCE CC-BY-4.0 ARCHIVED 2026-08-10T04:19:43Z SOURCE https://www.underworldcode.org/setting-up-underworld-dependencies/

```
    g++ \
    git
```

1.-.i. *Arch Linux:*

… and derived distributions (Manjaro)

```
sudo pacman -Syy
sudo pacman -S \
    make \
    gcc \
    gcc-fortran \
    python3 \
    git
```

1.a. Getting PETSc sources

The PETSc source code can be downloaded directly from the Web using git (Recommended)

```
git clone -b release https://gitlab.com/petsc/petsc.git petsc
```

```
cd petsc # Step in directory
git pull # Do a pull, just in case
git checkout v3.16.1 # Checkout vMAJOR_MINOR_PATCH
```

1.b. Configuring PETSc

1.b.i. *Note on External Packages:*

PETSc (the Portable, Extensible Toolkit for Scientific Computation) can be configured to work with multiple of tools. Here we detail one possible configuration that should be sufficient for most Underworld users. The PETSc configuration system gives you the option to download most of the external packages. We can thus leverage PETSc to install most of the tools we need…

Optimized BLAS:

PETSc has only one hard requirement which is the BLAS (Basic Linear Algebra Subprograms) library. BLAS is available on most systems but some packages provide optimized version of the libraries that are more efficient. In most cases we recommend using the OPENBLAS packages. The Intel Math Library (MKL) also provides optimized BLAS libraries, it is free and commonly available on HPC machines.

Like most external packages you can download OPENBLAS by passing the `--download-openblas` command to the configuration script. At the time of writing this causes some errors and we chose to install OPENBLAS using the package manager of out LINUX distribution.

Ubuntu:

Note that Ubuntu provides a `libopenblas-dev` package that depends on `libopenblas-serial-dev` OR `libopenblas-openmp-dev`. Use the serial version when possible.

```

```

```
sudo apt-get install \
    libopenblas-serial-dev \
    libopenblas-dev
```

Arch Linux:

```
sudo pacman -S openblas
```

1.b.ii. *MPI (Message Passive Interface):*

PETSc and Underworld use MPI. There are mainly 2 implementation of MPI:

- MPICH
- OPENMPI

The differences are irrelevant for most users. The important thing to keep in mind is that all packages used with PETSc must be configured to use the same MPI installation.
We discourage users to build PETSc againts the MPI installation provided by their LINUX distribution. This is because upgrading the package may cause PETSc to complain that it was built with a different version of the one available on the machine.
We recommend treating MPI as a normal External Package and have it installed via the PETSc configure script (see below).

1.b.iii. *Configuring and Installing:*

We are going to install PETSc in the `/opt` directory which is a rather standard location for built-from-sources packages.

From the PETSc source directory:

```
PETSC_VERSION=3.16.1 # Should Match version downloaded!

sudo mkdir /opt/petsc/${PETSC_VERSION}
sudo chown $USER /opt/petsc/${PETSC_VERSION}
```

A typical toolkit for Underworld includes some general utilities:

- HDF5 (for saving and reading H5 files)
- MPI (see above)
- MUMPS (**MU**ltifrontal **M**assively **P**arallel sparse direct **S**olver)
- METIS and ParMETIS (Parallel Graph Partitioning and Fill-reducing Matrix Ordering**)**
- SuperLU and SuperLU DIST (Supernodal LU Solver for sequential and distributed memory system)
- HYPRE (Scalable Linear Solvers and Multigrid Methods)
- ScaLAPACK (Scalable Linear Algebra PACKage)

```
cd petsc-${PETSC_VERSION}

./configure \
    --prefix=/opt/petsc/${PETSC_VERSION} \
```

```
    --with-debugging=yes                \
    --COPTFLAGS="-O3"                 \
    --CXXOPTFLAGS="-O3"               \
    --FOPTFLAGS="-O3"                 \
    --with-shared-libraries             \
    --with-cxx-dialect=C++11            \
    --with-make-np=8                    \
    --download-mpich=yes                \
    --download-hdf5=yes                 \
    --download-mumps=yes                \
    --download-parmetis=yes             \
    --download-metis=yes                \
    --download-superlu=yes              \
    --download-hypre=yes                \
    --download-superlu_dist=yes         \
    --download-scalapack=yes            \
    --download-cmake=yes
```

Note that in the above command, I am passing some optimization flags to the compilers for completeness (COPTFLAGS, CXXOPTFLAGS, FOPTFLAGS). However, I am also passing the `--with-debugging=yes` which effectively overwrite the flag options.

The configuration step should download all the required packages.

Once it is done you can build PETSc by doing:

```
make PETSC_DIR=$PWD PETSC_ARCH=arch-linux-c-debug all
```

Then install it to its destination location:

```
make PETSC_DIR=$PWD PETSC_ARCH=arch-linux-c-debug install
```

Do a final check:

```
make PETSC_DIR=/opt/petsc/${PETSC_VERSION} PETSC_ARCH="" check
```

Which should return:

```
Running check examples to verify correct installation
Using PETSC_DIR=/opt/petsc/3.16.1 and PETSC_ARCH=
C/C++ example src/snes/tutorials/ex19 run successfully with 1 MPI process
C/C++ example src/snes/tutorials/ex19 run successfully with 2 MPI processes
C/C++ example src/snes/tutorials/ex19 run successfully with hypre
C/C++ example src/snes/tutorials/ex19 run successfully with mumps
C/C++ example src/snes/tutorials/ex19 run successfully with superlu_dist
C/C++ example src/vec/vec/tests/ex47 run successfully with hdf5
Fortran example src/snes/tutorials/ex5f run successfully with 1 MPI process
Completed test examples
```

1.c. Conclusion

We have described one way to have a working PETSc installation to be used with Underworld. Experienced users may choose a different path and have different installations of PETSc pointing to different libraries with optimization options turned on etc… Users may also want to experiment with different compilers, different implementation of BLAS and/or MPI for example. PETSc is very powerful and flexible so many configurations can (should?) be tested.

Feel free to contact me or the other members of the team on Github (@rbeucher)

References

Beucher, R. (2022). *Configuring and Installing PETSc for Underworld*. https://doi.org/10.59350/t7ghx-8f823